Designing a Resource-Efficient Convolutional Neural Network for DDoS Detection in 5G IoT Environments

Anna Padée, Joachim Schmidt, Eryk Schiller
Institute of Industrial and IT Engineering (InTECH)
Geneva School of Engineering, Architecture and Landscape (HEPIA)
University of Applied Sciences and Arts of Western Switzerland (HESSO)
email: anna.padee@hes-so.ch

***Abstract*—Virgo is a cloud-native 5G security testbed. Its mobile operator (Open5GS, srsRAN), IoT application services, and machine-learning-based protection services run as containerized Cloud-Native Functions (CNFs). Real Mirai-derived malware is confined in isolated QEMU/KVM Virtual Machines (VMs). The resulting DDoS traffic traverses a live 5G data plane end-to-end. Virgo's original protection service uses CICFlowMeter for flow feature extraction and XGBoost for classification. In previous work, we showed that its per-flow state management causes a severe memory increase under randomized SYN-flood campaigns. This makes it unsuitable for resource-constrained edge accelerators such as FPGA-based SmartNICs. This paper extends Virgo with a packet-based alternative: a lightweight Convolutional Neural Network (CNN) that operates directly on windows of raw packet bytes, bypassing flow aggregation entirely. We establish the CNN architecture through a systematic hyperparameter search over 64 configurations. The selected model has approximately 2.5 million parameters and occupies approximately 2.5 MB under 8-bit quantization. It achieves 99.5% detection accuracy on testbed-captured Mirai traffic. Deployed as a CNF alongside the existing flow-based path, the CNN extends Virgo into a platform for developing, comparing, and deploying AI-driven cybersecurity services at the edge of 5G IoT networks.**

***Index Terms*—DDoS detection, convolutional neural networks, 5G networks, IoT security, Mirai botnet, packet-level classification, flow-based detection, XGBoost, model quantization, resource-efficient deep learning, network intrusion detection**

I. Introduction

Cloud-native architectures have transformed how network services are deployed, enabling operators to compose and scale network functions as lightweight containers. In the mobile domain, 5G core and radio-access components are increasingly instantiated as Cloud-Native Network Functions (CNFs), and this paradigm extends naturally to security: protection mechanisms can be modelled as cloud-native services deployed on demand within the same infrastructure. When a software container cannot deliver line-rate performance, the same service logic can be offloaded to an FPGA-based SmartNIC, realising a *hardware-accelerated CNF* that retains cloud-native orchestration while achieving data-plane-native throughput. In previous work [1], we presented Virgo, a fully cloud-native 5G security testbed. Its mobile operator (Open5GS, srsRAN), IoT traffic sources, and ML-based detection services run as CNFs. Real Mirai-derived malware is confined inside isolated QEMU/KVM virtual machines. Its Malware Management and Control (MMC) framework separates the control plane from the attack data plane, enabling reproducible SYN-flood campaigns whose traffic traverses the live 5G data plane end-to-end under strict containment. Virgo's original protection CNF implements the literature's dominant paradigm: containerized CICFlowMeter extracts over 80 flow-level features, and containerized XGBoost classifies them. We showed in [1] that this approach incurs an $\approx 7.6\times$ memory increase under randomised Mirai SYN floods, rendering it unsuitable for edge deployment on FPGA-based SmartNICs, where on-chip memory is measured in megabits. To address this, this paper adds a second protection CNF: a lightweight CNN operating directly on windows of raw packet bytes, bypassing flow aggregation entirely. Both protection CNFs are evaluated on identical traffic captured at the 5G user-plane interface. We establish the CNN's architecture through a systematic hyperparameter search over 64 configurations and evaluate post-training quantization (float16, int8). The resulting model achieves 99.5% accuracy with a footprint of approximately 2.5 MB under int8 quantisation. We deliberately constrained the model size to fit within FPGA platforms offering roughly 2.5 MB of on-chip memory, positioning it as a candidate for further hardware-oriented optimization and synthesis via hls4ml [2], [3].

The paper has the following organization. Section II reviews related work. Section III describes the architecture. Section IV details both protection CNFs. Section V presents results, including CICFlowMeter's memory bottleneck, the hyperparameter search outcome, and quantisation effects. Section VI concludes and outlines the path toward hardware-accelerated protection CNFs for 6G cloud infrastructures.

II. Related Work

This section reviews flow-based DDoS detection and its limitations, deep learning approaches that reduce or eliminate reliance on flow-level feature engineering, and IoT botnet testbed environments relevant to our platform Virgo.

A. Flow-Based Detection and Its Limitations

The prevailing paradigm for ML-based DDoS detection relies on statistical features extracted from bidirectional network flows. CICFlowMeter [4] is a widely adopted tool that produces over 80 statistical features per flow from raw PCAP captures. The CIC-IDS-2017 [5] and CIC-DDoS2019 [6] benchmark datasets, generated with this tool, have become standard evaluation targets. Tree-based classifiers such as XGBoost [7] routinely report accuracies exceeding 99% on these benchmarks [8]. Liu et al. [9] further demonstrate the use of XGBoost for DDoS detection in Named Data Networking.

However, the flow-based paradigm carries practical limitations. Flow aggregation requires maintaining per-flow state and delays classification until a sufficient number of packets have been accumulated or a configurable timeout expires, impeding real-time detection. The computational and memory overhead of the aggregation process itself becomes problematic under high-rate DDoS conditions with large numbers of concurrent micro-flows. Moreover, the resulting feature dimensionality and aggregation logic demand resources that exceed the capacity of hardware accelerators such as FPGA-based SmartNICs. Engelen et al. [10] additionally revealed significant errors in the CIC-IDS-2017 dataset's attack simulation, feature extraction, and labelling procedures, cautioning that high reported accuracies may partly reflect dataset artifacts. These limitations collectively motivate detection approaches that bypass flow-level feature extraction.

B. Deep Learning for DDoS Detection

Deep learning has been extensively applied to DDoS and network intrusion detection, using both flow-derived features and raw packet representations.

Flow-feature-based models. Halbouni et al. [11] combine CNN and LSTM with dropout and batch normalization, achieving 99.64% on CIC-IDS-2017. Jose and Jose [12] compare DNN, LSTM, and CNN on the same dataset, reporting 99.61% for CNN. Hnamte and Hussain [13] evaluate deep CNNs on multiple IDS datasets, achieving 99.96% accuracy. Yuan et al. [14] present DeepDefense, one of the earliest deep learning frameworks for DDoS. Elsayed et al. [15] introduce DDoSNet for multi-class attack detection. Najar and Manohar Naik [16] address CNN-based detection and mitigation in SDN. More recently, Wahab et al. [17] evaluate CNN, DNN, and Transformer architectures on the CIC-IoT-2023 dataset, achieving 99.0% accuracy for CNN in binary DDoS classification. In the 5G domain, Chen et al. [18] develop a CNN-BiLSTM model for private 5G networks.

Raw-packet and compact-representation models. An emerging direction investigates models that operate on raw packet bytes or on compact, fixed-size representations, thereby eliminating the flow aggregation bottleneck. Soltani et al. [19] apply an LSTM to raw byte sequences, achieving 99.8% recall on the CIC-IDS-2017 dataset. Yu et al. [20] propose PBCNN, a hierarchical CNN that first extracts features from the raw bytes of individual packets, then constructs session-level representations from the sequence of per-packet features, achieving strong results on CIC-IDS-2017 and CSE-CIC-IDS-2018. Doriguzzi-Corin et al. [21] present LUCID, a lightweight CNN for DDoS detection that converts short flow segments into compact, fixed-size tensors, achieving a 40× reduction in processing time; in subsequent work [22] they extend packet-level analysis to programmable data planes.

Ogonowski et al. [23] stack raw packets into 2D windows (150 packets × 351 bytes) and classify each packet individually using FCNN, CNN, CNN-LSTM, and EfficientNet architectures. Their saliency-map analysis reveals that FCNN decisions depend primarily on header features, whereas CNN and hybrid models attend to both headers and payloads, suggesting better generalization. Their randomized replacement of MAC addresses, IPs, and ports prevents learning of spurious identity features while preserving inter-packet dependencies.

Table I summarises representative detection approaches across both paradigms.

TABLE I: Representative DDoS/intrusion detection methods.

Method	Input	Classif.	Dataset	Acc. [%]
XGBoost [8]	Flow feat.	Flow	CIC-IDS-17	99.65
DCNN [13]	Flow feat.	Flow	CIC-IDS-17	99.96
CNN [12]	Flow feat.	Flow	CIC-IDS-17	99.61
CNN-LSTM [11]	Flow feat.	Flow	CIC-IDS-17	99.64
DDoSNet [15]	Flow feat.	Flow	CIC-DDoS19	99.0
CNN-BiLSTM [18]	Flow feat.	Flow	Private 5G	99.35[1]
CNN/DNN/Tr. [17]	Flow feat.	Flow	CIC-IoT-23	99.0
DID (LSTM) [19]	Raw bytes	Window	CIC-IDS-17	99.8
PBCNN [20]	Raw bytes	Hier.	CIC-IDS-17	99.99[1]
LUCID [21]	Compact	Flow seg.	Multiple	98.9–99.9
FCNN [23]	Raw pkts	Packet	CIC-IDS-17	99.93
CNN (2D) [23]	Raw pkts	Packet	CIC-IDS-17	98.77
CNN-LSTM [23]	Raw pkts	Packet	CIC-IDS-17	98.85
EfficientNet [23]	Raw pkts	Packet	CIC-IDS-17	99.17
Ours (CNN)	**Raw pkts**	**Packet**	**Virgo (Mirai)**	**99.5**

C. IoT Botnet Testbeds

Mirai and its descendants remain the dominant IoT-botnet threat [24], [25]. Existing testbeds span a wide fidelity spectrum. Fully virtualised environments such as the GNS3-based laboratory of Sáez-de-Cámara et al. [26], Bot-IoT [27], and TON_IoT [28] emphasise reproducibility but do not capture device firmware or wireless characteristics. Application-layer generators such as IoT-Flock [29] and MQTT-focused datasets [30] produce protocol-level traffic but share the same limitation. Hardware testbeds offer more authentic behaviour: N-BaIoT [31] infects commercial devices with Mirai and BashLite, MedBIoT [32] combines real and emulated devices, and Edge-IIoTset [33] provides large-scale IoT/IIoT datasets. Comparatively few platforms incorporate a cellular data plane: 5GReplay [34] applies rule-based mutation to 5G traffic, and Carrega et al. [35] present a modular B5G/6G security testbed. Our platform differs in that it executes *real* Mirai-derived malware inside isolated VMs on Raspberry Pi 5 devices, with traffic traversing an end-to-end 5G data plane,

[1] Value calculated from the results in the paper

thereby combining device-level authenticity, cellular realism, and reproducibility. The design of CNN architectures with computational costs compatible with future FPGA deployment via the hls4ml framework [2], [3] is a central consideration of the present work.

III. Architecture

A. Testbed Architecture

Figure 1 depicts our Virgo testbed, which is used to generate the training and evaluation data for the proposed CNN-based DDoS detector. The infrastructure comprises two network segments interconnected through a central router that simultaneously provides DHCP, firewall, and packet-capture services. The wired segment (NETA, 192.168.10.0/24) hosts three Raspberry Pi nodes running containerised application services, i.e., an HTTP server (nginx), a video server (mediamtx), and an MQTT broker (Mosquitto), that produce reproducible legitimate traffic patterns. The mobile segment (NETB, 192.168.20.0/24) is anchored by a containerized srsRAN gNB connected to a USRP B210 SDR and an Open5GS core network, operating in 5G Standalone mode on band N77. All over-the-air experiments were conducted under a test license issued by the Swiss Federal Office of Communications (OFCOM). Two categories of wireless devices attach to this segment. Legitimate endpoints, consisting of IoT-class Raspberry Pi 5 devices generating MQTT, HTTP, UDP, and video traffic, alongside a human-operated smartphone (Samsung S23) producing realistic user-driven activity; and an infected endpoint, where a Raspberry Pi 5 hosts a QEMU/KVM virtual machine executing real Mirai-derived malware [36]. The malicious VM is orchestrated via the Malware Management and Control (MMC) framework, which separates the control plane from the attack data plane through AF_UNIX sockets (mmc-host-daemon), virtio serial links (mmc-vm-daemon), and IPv6 multicast discovery (mmc-cli), ensuring strict containment while enabling reproducible attack execution [1]. All user-plane traffic, both legitimate and malicious, is captured on the router using tcpdump/wireshark, yielding raw PCAP traces with IP-based ground-truth labelling. Using this setup, a total of 800 MB of traffic was collected. These traces feed two parallel detection paths. Path A implements the conventional flow-based approach [1]: a containerized CICFlowMeter instance converts PCAPs into bidirectional flow records with over 80 statistical features, which are then classified by a containerized XGBoost model. Path B, the focus of this paper, bypasses flow aggregation entirely: a sliding-window preprocessing stage constructs compact, fixed-size representations directly from raw packets, which are then classified by a lightweight CNN whose architecture is determined through systematic hyperparameter search. Path A supports binary or multi-class classification, whereas Path B performs binary (benign vs. DDoS) per-packet classification. The CNN architecture is explicitly designed with resource constraints compatible with future deployment on an FPGA-based SmartNIC via the hls4ml synthesis framework. The central contribution of this paper is the design, training, and evaluation of the CNN model in Path B: we present the hyperparameter exploration that determines the network architecture, report detection accuracy on locally captured Mirai traces, and quantify the computational cost of inference, demonstrating that competitive detection performance can be achieved with a dramatically reduced resource footprint compared to the flow-based baseline.

IV. Detection Pipelines

Starting from the same raw PCAP captures collected at the observation point (Sect. III), we evaluate two parallel detection pipelines that differ fundamentally in how they represent network traffic for classification. Path A follows the dominant paradigm in the literature: a flow-based approach that aggregates packets into bidirectional flows, extracts statistical features, and classifies them with a gradient-boosted tree ensemble. Path B bypasses flow aggregation entirely, operating directly on raw packet bytes organised into fixed-size sliding windows and classified by a lightweight convolutional neural network. The dual-pipeline design enables controlled comparison of detection accuracy and computational cost on identical traffic traces processed through two independent paths.

A. Path A: Flow-Based Detection from [1] (CICFlowMeter + XGBoost)

Captured traffic is processed by a fully containerized pipeline to guarantee reproducibility and portability. Raw PCAPs are converted into bidirectional flow records by CICFlowMeter V3[1] [4], which holds per-flow state until a flow terminates (via mutual FIN exchange in TCP) or a configurable timeout elapses, yielding over 80 statistical features per flow covering duration, inter-arrival times, byte and packet counts, and burst metrics. The resulting records are exported in CSV and Parquet[2] formats, then cleaned, label-encoded, and feature-scaled. CICFlowMeter runs inside a dedicated Docker container with a Java heap of 8 GB; because its memory scales with the number of concurrently open flows, Mirai SYN-flood campaigns with randomized destination addresses and ports can exhaust memory outright. We therefore split captures into chunks of at most 64 MB and employ a three-pass retry mechanism. This bottleneck is quantified in Sect. V-A. XGBoost was selected for Path A because the flow-based representation consists primarily of structured, heterogeneous tabular features, for which gradient-boosted decision trees are particularly well suited. Compared with models that require more extensive feature scaling or architecture design, XGBoost can directly capture nonlinear relationships and interactions among flow-level features while maintaining relatively low training and inference complexity. In addition, its feature-importance mechanisms provide useful interpretability for a security-oriented application.

Classification is performed by XGBoost [7], configured with 100 estimators, maximum depth 6, learning rate 0.1, subsampling and column-subsampling ratios of 0.8, the histogram-

[1]https://github.com/ahlashkari/CICFlowMeter

[2]https://parquet.apache.org/

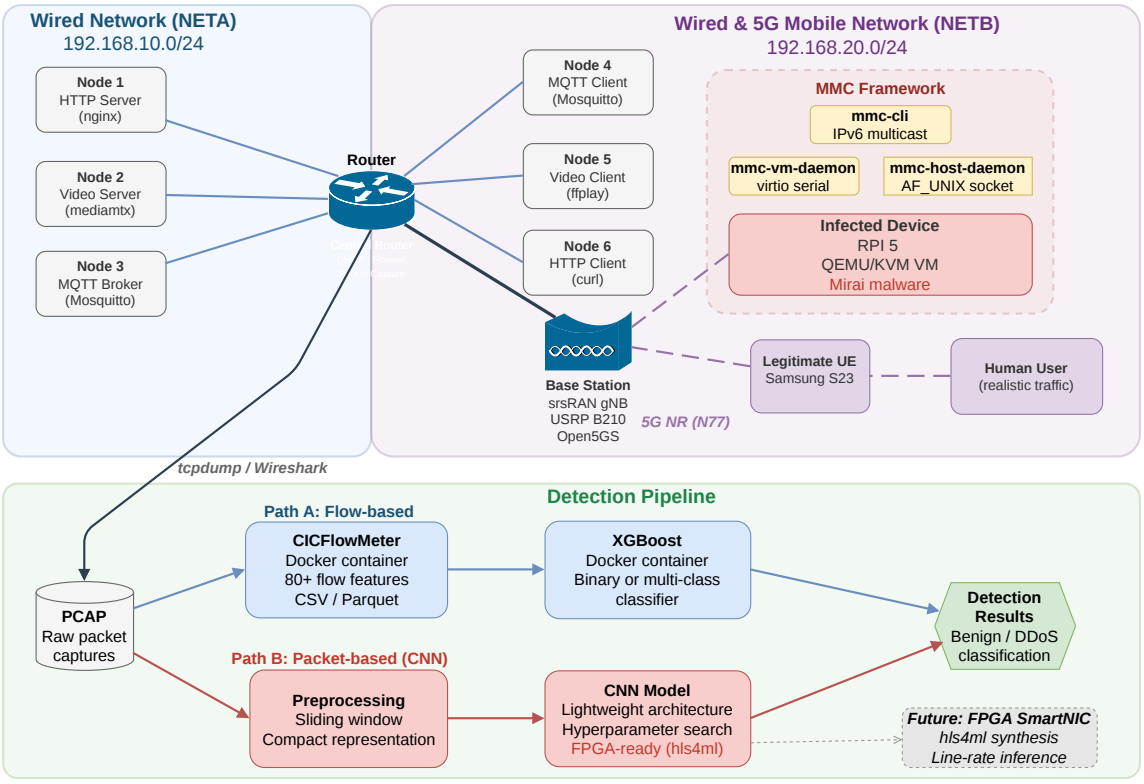


Fig. 1: Integrated cellular–IoT testbed.

based tree method, and binary log-loss. The model is trained on locally generated Virgo traces using a 50/50 train/test split, so that both training and evaluation data originate from the same testbed environment. Training and inference run in a dedicated Python container; hardware acceleration is not yet employed for Path A.

B. Path B: Packet-Based Detection (Sliding-Window CNN)

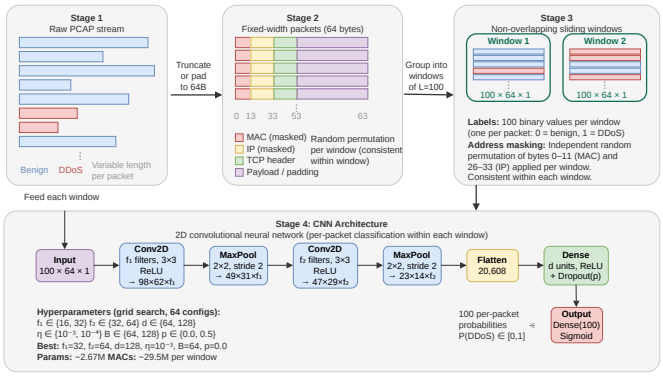


Fig. 2: Path B: sliding-window preprocessing and CNN architecture.

Path B replaces the entire flow-extraction stage with a lightweight CNN that operates directly on raw packet bytes, organised into fixed-size sliding windows (cf. Fig. 2). The pipeline proceeds in five steps: PCAP-to-array conversion, window construction with address masking, CNN architecture definition, hyperparameter search, and selection of the final architecture.

1) PCAP-to-array conversion: Raw PCAPs are read packet by packet using Scapy, and each packet is serialised to its byte-level representation. Since packets vary in length, i.e., from short TCP SYNs of approximately 54 bytes to full-payload HTTP responses exceeding 1500 bytes, each packet is either truncated or zero-padded to a fixed width of $W = 64$ bytes. This value preserves the complete protocol header stack (Ethernet 14 B + IP 20 B + TCP 20 B = 54 B) plus approximately 10 bytes of TCP options or payload, while reducing the input data volume to roughly 4% of a full 1500-byte MTU packet. Labeling is performed at the packet level based on source and destination IP addresses: any packet whose destination IP address falls within the targeted address range and originates from a DDoS capture file is labeled as attack (1); all other packets are labeled as benign (0). The result is an HDF5 dataset containing an array $\mathbf{X} \in \mathbb{R}^{N\times 64}$ of unsigned 8-bit byte vectors and a binary label vector $\mathbf{y} \in \{0,1\}^N$.

2) Sliding-window construction and address masking: The per-packet array is partitioned into non-overlapping windows of $L = 100$ consecutive packets, producing three-dimensional tensors of shape $(100 \times 64 \times 1)$ suitable for 2D convolutional processing. Each window captures a temporal snapshot of 100 consecutive packets as they appeared on the wire, preserving

inter-packet ordering within the window boundary. The corresponding labels are grouped identically, yielding a vector of 100 binary labels per window, i.e., the model performs per-packet classification within the temporal context of the surrounding window.

To prevent the CNN from learning spurious associations with specific MAC or IP addresses, an address masking procedure is applied independently to each window. A random permutation of the byte value space $\{0, \ldots, 255\}$ is generated per window and applied consistently to the byte positions corresponding to source and destination MAC addresses (bytes 0–11) and IP addresses (bytes 26–33). This *randomised replacement* preserves structural relationships between packets within the same window, i.e., two packets sharing the same source IP retain the same transformed value, while ensuring that absolute address values carry no discriminative information across windows. The approach follows the principle introduced by Ogonowski et al. [23], who demonstrated that randomised replacement outperforms simple address removal by maintaining inter-packet dependencies. The preprocessed dataset is stored in HDF5 format and split into training and test sets using a 67/33 stratified random split.

3) CNN architecture: The detection model is a 2D CNN implemented in TensorFlow/Keras that treats each $(100 \times 64 \times 1)$ window as a single-channel image:

1) **First convolutional stage:** Conv2D with f_1 filters of kernel size 3×3 and ReLU activation, followed by 2×2 max-pooling with stride 2, producing feature maps of shape $(49 \times 31 \times f_1)$.
2) **Second convolutional stage:** Conv2D with f_2 filters of kernel size 3×3 and ReLU activation, followed by 2×2 max-pooling, producing feature maps of shape $(23 \times 14 \times f_2)$.
3) **Classification head:** the pooled feature maps are flattened into a vector of $23 \times 14 \times f_2$ elements, passed through a Dense layer with d units and ReLU activation, a Dropout layer with rate p, and a final Dense layer with 100 sigmoid-activated outputs—one per packet in the window.

The output layer produces 100 independent probabilities $P(\text{DDoS}) \in [0, 1]$, binarised at a threshold of 0.5. The model is compiled with the Adam optimiser and trained using binary crossentropy loss with accuracy as the monitoring metric.

4) Hyperparameter search: We establish the architecture through a systematic grid search over six hyperparameters, yielding $2^6 = 64$ experimental configurations (Table II). The number of training epochs was fixed at $E = 10$ for all configurations. Each configuration is trained from random initialisation and evaluated on the held-out test set. For every run, model weights, architecture specification, training history, multilabel confusion matrix, and per-class classification report are persisted. A global summary aggregates macro-averaged precision, recall, F1-score, best validation accuracy, and best validation loss across all experiments.

5) Selected architecture: The best-performing configuration uses $f_1 = 32$, $f_2 = 64$, $d = 128$, $\eta = 10^{-3}$, $B = 64$, and $p = 0.0$. After the second pooling stage, the flattened vector contains $23 \times 14 \times 64 = 20{,}608$ elements. Table III details the layer-by-layer structure.

TABLE II: Hyperparameter search space ($2^6 = 64$ configurations).

Hyperparameter	Values
Filters, 1st conv (f_1)	16, 32
Filters, 2nd conv (f_2)	32, 64
Dense units (d)	64, 128
Learning rate (η)	10^{-3}, 10^{-4}
Batch size (B)	64, 128
Dropout rate (p)	0.0, 0.5
Epochs (E)	10

TABLE III: Selected CNN architecture (best hyperparameter configuration).

Layer	Output shape	Kernel	Parameters
Input	$(100, 64, 1)$	—	0
Conv2D + ReLU	$(98, 62, 32)$	3×3	320
MaxPooling2D	$(49, 31, 32)$	2×2	0
Conv2D + ReLU	$(47, 29, 64)$	3×3	18,496
MaxPooling2D	$(23, 14, 64)$	2×2	0
Flatten	(20,608)	—	0
Dense + ReLU	(128)	—	2,637,952
Dense + Sigmoid	(100)	—	12,900
Total trainable parameters			**2,669,668**

The ≈ 2.5 million parameters are dominated (98.8%) by the Dense layer connecting the 20,608-element flattened vector to 128 units. This parameter count is well within the synthesis capacity of the hls4ml framework [3] for FPGA deployment and is substantially smaller than EfficientNet-based alternatives evaluated by Ogonowski et al. [23]. Inference requires approximately 29.5M multiply–accumulate (MAC) operations per 100-packet window, i.e., about 0.3M MACs per packet.

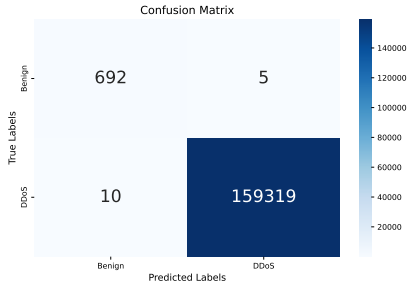
V. Results

A. Path A: Flow-Based Detection from [1] (CICFlowMeter + XGBoost)

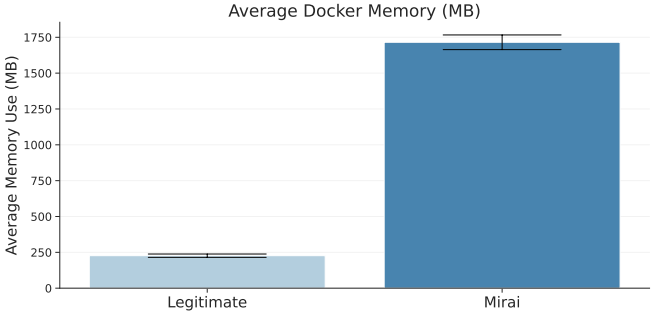
For completeness, we summarise the flow-based results previously reported in [1], which serve as the baseline for the CNN evaluated in Sect. V-B. The XGBoost classifier described in Sect. IV-A, trained on CICFlowMeter-extracted flow features, achieved strong detection performance on the held-out test set: out of 160,026 flows, only 5 benign flows are misclassified as DDoS and 10 attack flows are missed (Fig. 3a), corresponding to an overall accuracy of 99.99%, a macro F1 of 0.995, a false-positive rate of 0.72%, and a false-negative rate of 0.006%. At the flow level, this matches or exceeds the packet-level performance of the CNN reported in Sect. V-B1.

However, this detection performance comes at a substantial and disproportionate memory cost. Fig. 3b, reproduced from [1], shows the average Docker container memory consumption of the CICFlowMeter extraction stage, measured

separately over captures of legitimate traffic and of Mirai DDoS traffic. Legitimate traffic is processed with a modest Docker memory footprint of $\approx$ 226 MB on average. Under Mirai attack traffic, however, average memory consumption rises to $\approx$ 1,717 MB, i.e., a $\approx 7.6\times$ increase, and approaches the 8 GB Java heap allocated to the container (Sect. IV-A) during the largest captures. This disproportionate growth is a direct consequence of the flow-aggregation paradigm itself: Mirai SYN-flood traffic with destination addresses randomized within a target range and randomized ports generates a very large number of short-lived, single-packet flows that never terminate via a mutual FIN exchange, forcing CICFlowMeter to retain per-flow state for each one until its timeout elapses. Because memory scales with the number of concurrently open flows rather than with byte volume, attack traffic, i.e., the traffic a DDoS detector must process in real time, is also the traffic that stresses the flow-extraction pipeline the most, motivating the chunking and retry mechanisms described in Sect. IV-A and, more fundamentally, motivating the packet-based Path B approach evaluated in the remainder of this section.



(a) Confusion matrix (XGBoost, flow-level).



(b) CICFlowMeter memory use by traffic type, run in a Docker container.

Fig. 3: Path A (flow-based) results: (a) confusion matrix of the XGBoost classifier on held-out flows, and (b) average Docker container memory consumption of the CICFlowMeter extraction stage for legitimate versus Mirai traffic, with error bars. [1]

B. Path B: Packet-Based Detection (Sliding-Window CNN)

This section reports the outcome of the hyperparameter search described in Sect. IV-B4, quantifies the effect of post-training quantization on model size and detection performance, and characterises the accuracy–footprint trade-off relevant to future FPGA deployment.

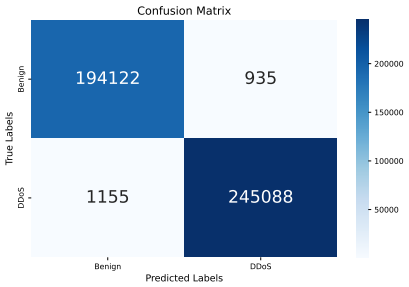
1) Hyperparameter Search Outcome: Across the 64 grid-search configurations (Table II), macro-averaged F1 scores span a wide range, from 0.285 to 0.995 (median 0.57), reflecting substantial sensitivity of the sliding-window CNN to hyperparameter choice. Rather than varying continuously, F1 scores cluster into three well-separated performance tiers: a high-performing tier (F1 $\gtrsim$ 0.94), a mid-performing tier (F1 $\approx$ 0.65–0.85), and a low-performing tier (F1 $\lesssim$ 0.48), visible in Fig. 4b and consistently reproduced in Fig. 5a. Precision (median 0.50, IQR 0.30–0.96) is consistently lower than recall (median 0.69, IQR 0.54–0.95) across the search, indicating that the majority of configurations favour minimising false negatives at the expense of false positives—a desirable bias for a DDoS detector, where missed attack packets are typically more costly than false alarms. Validation loss follows a correspondingly broad distribution (median 0.33, range 0.03–0.51).

The best-performing configuration uses $f_1 = 32$ filters in the first convolutional stage, $f_2 = 64$ filters in the second, a Dense layer of $d = 128$ units, learning rate $\eta = 10^{-3}$, batch size $B = 64$, dropout rate $p = 0.0$, and was trained for 10 epochs. It falls within the high-performing tier identified above, achieving a macro F1 of 0.995, macro precision of 0.995, and macro recall of 0.995 on the held-out test set, with an overall packet-level accuracy of 99.53%. Fig. 4a reports the corresponding confusion matrix: out of 441,300 test packets, only 935 benign packets are misclassified as DDoS (false-positive rate 0.48%) and 1,155 attack packets are missed (false-negative rate 0.47%), confirming that the selected architecture attains a favourable, well-balanced operating point rather than trading recall for precision or vice versa.

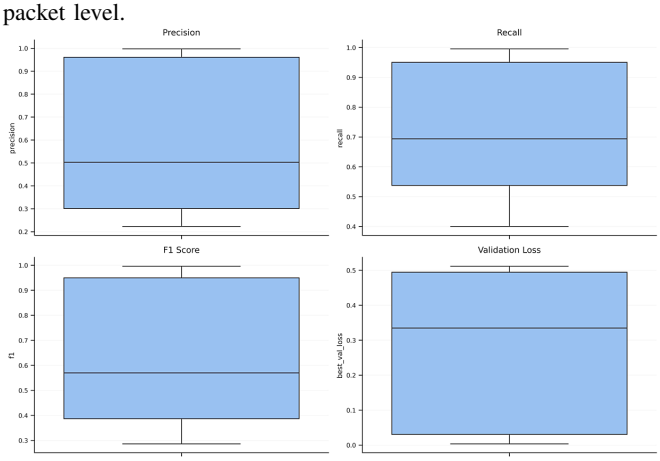
2) Effect of Post-Training Quantization: To assess deployability under the memory constraints of an FPGA-based SmartNIC, we also exported each trained model under two post-training quantization schemes, `float16` and `int8`, alongside the unquantized (`none`) baseline.

Model size. Quantization substantially reduces the on-disk footprint (Fig. 5b). Unquantized models occupy $\approx$ 10.2, 5.1, or 2.5 MB depending on f2 and d, which dominate the parameter count. `float16` and `int8` quantization halve these figures successively (int8: $\approx$ 2.5, 1.3, and 0.7 MB), an overall $\approx$ 4× compression; the selected model occupies $\approx$ 2.5 MB under `int8`.

Detection performance. Despite this size reduction, macro F1 is essentially unaffected by quantization (Fig. 5a): the median F1 across all 64 configurations is 0.57 for `none`, `float16`, and `int8` alike, and the three performance tiers identified in Sect. V-B1 are preserved under both quantization schemes, with only minor, non-systematic per-configuration variation. Fig. 5 confirms this jointly: for a given hyperparameter configuration, the `float16` and `int8` variants retain the same F1 tier as their unquantized counterpart while occupying a fraction of the memory. Notably, several `int8` configurations reach the high-performing tier (F1 $\gtrsim$ 0.94)

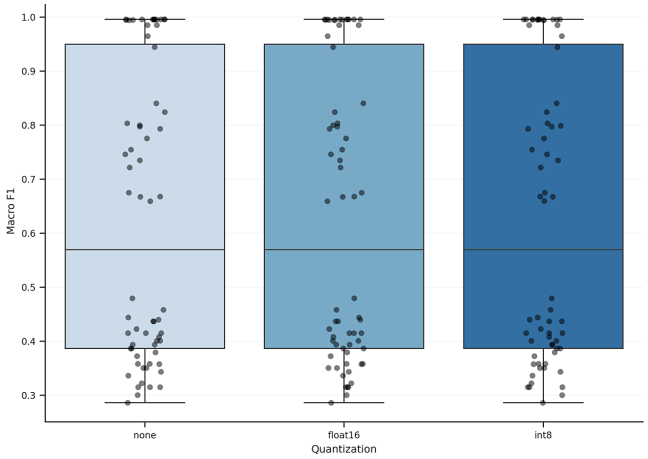


(a) Confusion matrix of the CNN architecture, aggregated at the packet level.

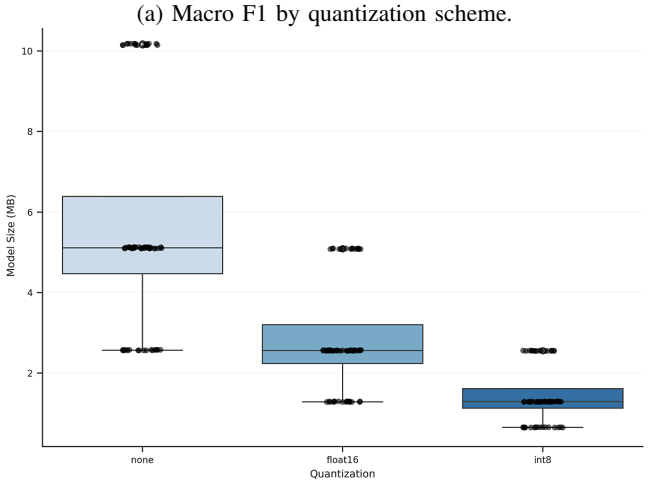


(b) Precision, recall, F1, and validation loss across the search.

Fig. 4: (a) Confusion matrix of the selected CNN architecture on the held-out test set, and (b) distribution of precision, recall, F1 score, and validation loss across the 64 hyperparameter configurations.



(a) Macro F1 by quantization scheme.



(b) Model size by quantization scheme.

Fig. 5: Effect of post-training quantization (`none`, `float16`, `int8`) on (a) detection performance and (b) model size, across the 64 hyperparameter configurations.

at approximately one quarter of the size of the largest unquantized models, indicating that quantization is a favourable, near cost-free lever for reducing the model's resource footprint ahead of hls4ml synthesis.

VI. Summary

This paper extended Virgo [1], our fully cloud-native 5G security testbed, with a packet-based protection CNF. In previous work, we showed that Virgo's flow-based path (CICFlowMeter + XGBoost), while achieving high in-domain accuracy, incurs an $\approx 7.6\times$ increase in memory usage under randomized Mirai SYN floods, i.e., a bottleneck that precludes deployment on resource-constrained edge hardware. The packet-based path introduced here replaces flow aggregation with a lightweight CNN operating on windows of raw packet bytes, evaluated on the same testbed-captured traffic. Across 64 hyperparameter configurations, we showed that detection performance is governed by architecture and training choices rather than numerical precision: int8 quantization reduces model size by roughly $4\times$ with negligible impact on macro F1. The selected architecture achieves 99.53% accuracy (macro F1 0.995) with balanced false-positive and false-negative rates, at a model size of approximately 2.5 MB, compared with the ≈ 1.7 GB average memory consumption of CICFlowMeter under attack traffic with 64 MB traffic chunks. The model size was deliberately constrained to fit within FPGA platforms offering roughly 2.5 MB of on-chip memory, motivating a compact architecture suitable for future hardware-oriented optimization and synthesis via hls4ml. Future work will complete this synthesis and deploy the accelerated protection CNF on an FPGA-based SmartNIC within the Virgo data plane, targeting inline DDoS detection at line rate directly in the 5G data plane. We further plan to extend the platform to support programmable, on-demand security services for emerging 6G cloud infrastructures.

Acknowledgements

This work was carried out within the framework of the NATWORK project—Net-Zero Self-Adaptive Activation of Distributed Self-Resilient Augmented Services. NATWORK has received funding from the European Union's Horizon Europe research and innovation programme through the Smart

Networks and Services Joint Undertaking (SNS JU) under Grant Agreement No. 101139285, with additional support from the Swiss State Secretariat for Education, Research and Innovation (SERI) and UK Research and Innovation (UKRI). The authors gratefully acknowledge this support.

REFERENCES

[1] J. Schmidt, L. Padial, A. Padee, and E. Schiller, "Virgo: On-Demand Protection in 5G Networks Through a Reproducible Jamming and Malware IoT Testbed," *IEEE International Conference on Cloud Networking*, 2026.

[2] J. Duarte *et al.*, "Fast inference of deep neural networks in FPGAs for particle physics," *Journal of Instrumentation*, vol. 13, no. 07, p. P07027, 2018.

[3] T. Aarrestad *et al.*, "Fast convolutional neural networks on FPGAs with hls4ml," *Machine Learning: Science and Technology*, vol. 2, no. 4, p. 045015, 2021.

[4] A. H. Lashkari, G. Draper Gil, M. S. I. Mamun, and A. A. Ghorbani, "Characterization of Tor traffic using time-based features," in *Proceedings of the 3rd International Conference on Information Systems Security and Privacy (ICISSP)*, 2017, pp. 253–262.

[5] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)*, vol. 1, 2018, pp. 108–116.

[6] Canadian Institute for Cybersecurity, "CIC-DDoS2019 dataset," https://www.unb.ca/cic/datasets/ddos-2019.html, 2019, accessed: 2026-07-14.

[7] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.

[8] M. A. Talukder, M. M. Islam, M. A. Uddin, K. F. Hasan, S. Sharmin, S. A. Alyami, and M. A. Moni, "Machine learning-based network intrusion detection for big and imbalanced data using oversampling, stacking feature embedding and feature extraction," *Journal of Big Data*, vol. 11, p. 33, 2024.

[9] L. Liu, W. Yu, Z. Wu, and S. Peng, "XGBoost-based detection of DDoS attacks in named data networking," *Future Internet*, vol. 17, no. 5, p. 206, 2025.

[10] G. Engelen, V. Rimmer, and W. Joosen, "Troubleshooting an intrusion detection dataset: The CICIDS2017 case study," in *2021 IEEE Security and Privacy Workshops (SPW)*, 2021, pp. 7–12.

[11] A. Halbouni, T. S. Gunawan, M. H. Habaebi, M. Halbouni, M. Kartiwi, and R. Ahmad, "CNN-LSTM: Hybrid deep neural network for network intrusion detection system," *IEEE Access*, vol. 10, pp. 99 837–99 849, 2022.

[12] J. Jose and D. V. Jose, "Deep learning algorithms for intrusion detection systems in Internet of Things using CIC IDS-2017 dataset," *International Journal of Electrical and Computer Engineering*, vol. 13, no. 1, pp. 1134–1141, 2023.

[13] V. Hnamte and J. Hussain, "Dependable intrusion detection system using deep convolutional neural network: A novel framework and performance evaluation approach," *Telematics and Informatics Reports*, vol. 11, p. 100077, 2023.

[14] X. Yuan, C. Li, and X. Li, "DeepDefense: Identifying DDoS attack via deep learning," in *2017 IEEE International Conference on Smart Computing (SMARTCOMP)*, 2017, pp. 1–8.

[15] M. S. Elsayed, N.-A. Le-Khac, S. Dev, and A. D. Jurcut, "DDoSNet: A deep-learning model for detecting network attacks," in *2020 IEEE 21st International Symposium on A World of Wireless, Mobile and Multimedia Networks (WoWMoM)*, 2020, pp. 391–396.

[16] A. A. Najar and S. M. Naik, "Cyber-secure SDN: A CNN-based approach for efficient detection and mitigation of DDoS attacks," *Computers & Security*, vol. 139, p. 103716, 2024.

[17] S. A. Wahab, S. Sultana, N. Tariq, M. Mujahid, J. A. Khan, and A. Mylonas, "A multi-class intrusion detection system for DDoS attacks in IoT networks using deep learning and transformers," *Sensors*, vol. 25, no. 15, p. 4845, 2025.

[18] C.-L. Chen, Y.-J. Rao, and C.-H. Sung, "Mitigating DDoS attacks in private 5G networks using deep learning," *Mobile Networks and Applications*, vol. 30, pp. 508–527, 2025.

[19] M. Soltani, M. Jafari Siavoshani, and A. H. Jahangir, "A content-based deep intrusion detection system," *International Journal of Information Security*, vol. 21, no. 3, pp. 547–562, 2022.

[20] L. Yu, J. Dong, L. Chen, M. Li, B. Xu, Z. Li, L. Qiao, L. Liu, B. Zhao, and C. Zhang, "PBCNN: Packet bytes-based convolutional neural network for network intrusion detection," *Computer Networks*, vol. 194, p. 108117, 2021.

[21] R. Doriguzzi-Corin, S. Millar, S. Scott-Hayward, J. Martínez-del Rincón, and D. Siracusa, "LUCID: A practical, lightweight deep learning solution for DDoS attack detection," *IEEE Transactions on Network and Service Management*, vol. 17, no. 2, pp. 876–889, 2020.

[22] R. Doriguzzi-Corin, L. A. D. Knob, L. Mendozzi, D. Siracusa, and M. Savi, "Introducing packet-level analysis in programmable data planes to advance network intrusion detection," *Computer Networks*, vol. 239, p. 110162, 2024.

[23] A. Ogonowski, M. Żebrowski, A. Ćwiek, T. Jarosiewicz, K. Klimaszewski, A. Padee, P. Wasiuk, and M. Wójcik, "Preliminary study on artificial intelligence methods for cybersecurity threat detection in computer networks based on raw data packets," *Computer Science*, vol. 26, no. SI, Jul. 2025.

[24] M. Gelgi, Y. Guan, S. Arunachala, S. S. R. Maddi, and N. Dragoni, "Systematic literature review of IoT botnet DDoS attacks and evaluation of detection techniques," *Sensors*, vol. 24, no. 11, p. 3571, 2024.

[25] A. Famera, B. Hilger, S. Bhunia, and P. Heil, "Analyzing the mirai IoT botnet and its recent variants: Satori, Mukashi, Moobot, and Sonic," 2025.

[26] X. Sáez-de Cámara, J. L. Flores, C. Arellano, A. Urbieta, and U. Zurutuza, "Gotham testbed: A reproducible IoT testbed for security experiments and dataset generation," *IEEE Transactions on Dependable and Secure Computing*, vol. 21, no. 1, pp. 186–203, 2024.

[27] N. Koroniotis, N. Moustafa, E. Sitnikova, and B. Turnbull, "Towards the development of a realistic botnet dataset in the Internet of Things for network forensic analytics: Bot-IoT dataset," *Future Generation Computer Systems*, vol. 100, pp. 779–796, 2019.

[28] N. Moustafa, "A new distributed architecture for evaluating AI-based security systems at the edge: Network TON_IoT datasets," *Sustainable Cities and Society*, vol. 72, p. 102994, 2021.

[29] S. Ghazanfar, F. Hussain, A. U. Rehman, U. U. Fayyaz, F. Shahzad, and G. A. Shah, "IoT-Flock: An open-source framework for IoT traffic generation," in *2020 International Conference on Emerging Trends in Smart Technologies (ICETST)*, 2020, pp. 1–6.

[30] H. Hindy, E. Bayne, M. Bures, R. Atkinson, C. Tachtatzis, and X. Bellekens, "Machine learning based IoT intrusion detection system: An MQTT case study (MQTT-IoT-IDS2020 dataset)," in *Selected Papers from the 12th International Networking Conference*. Springer, 2021, pp. 73–84.

[31] Y. Meidan, M. Bohadana, Y. Mathov, Y. Mirsky, A. Shabtai, D. Breitenbacher, and Y. Elovici, "N-BaIoT—network-based detection of IoT botnet attacks using deep autoencoders," *IEEE Pervasive Computing*, vol. 17, no. 3, pp. 12–22, 2018.

[32] A. Guerra-Manzanares, J. Medina-Galindo, H. Bahsi, and S. Nõmm, "MedBIoT: Generation of an IoT botnet dataset in a medium-sized IoT network," in *Proceedings of the 6th International Conference on Information Systems Security and Privacy (ICISSP)*, 2020, pp. 207–218.

[33] M. A. Ferrag, O. Friha, D. Hamouda, L. Maglaras, and H. Janicke, "Edge-IIoTset: A new comprehensive realistic cyber security dataset of IoT and IIoT applications for centralized and federated learning," *IEEE Access*, vol. 10, pp. 40 281–40 306, 2022.

[34] Z. Salazar, F. Zaïdi, H.-N. Nguyen, W. Mallouli, A. R. Cavalli, and E. Montes De Oca, "A network traffic mutation based ontology, and its application to 5G networks," *IEEE Access*, vol. 11, pp. 43 925–43 944, 2023.

[35] A. Carrega, F. Davoli, and R. Rabbani, "B5G/6G cyber security testbed," in *2025 33rd International Symposium on Modeling, Analysis, and Simulation of Computer and Telecommunication Systems (MASCOTS)*. IEEE, 2025, pp. 1–8.

[36] "Github - jgamblin/mirai-source-code: Leaked mirai source code for research/ioc development purposes," https://github.com/jgamblin/Mirai-Source-Code, [Online].